Python Fundamentals - Part 3

{ik} INTERVIEW KICKSTART

Agenda

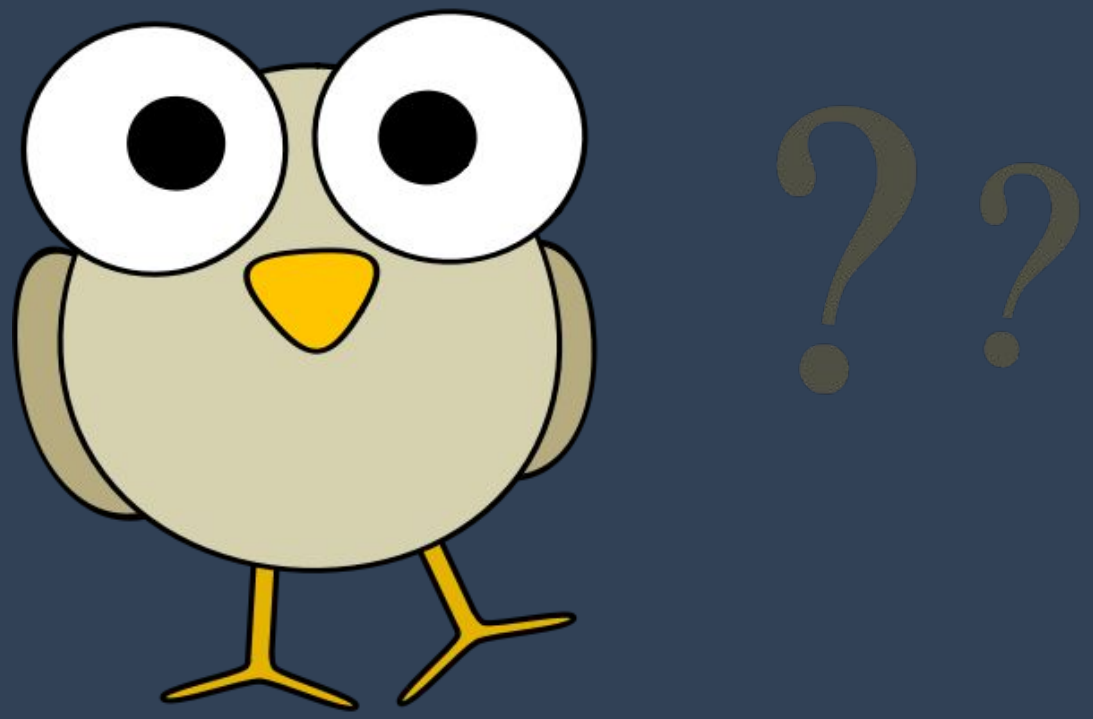
- Python OOP Concepts
- Scope of Variables in Python
- Custom Sort Functions in Python
- Exception Handling
- File Handling
- JSON module in Python

Python OOP Concepts

- What is OOP?
  - A programming paradigm that uses objects to represent and manipulate data
  - Focus on creating reusable and modular code

Python OOP Concepts

- Benefits of OOP in Python
  - Easier maintenance
  - Code reusability
  - Faster development



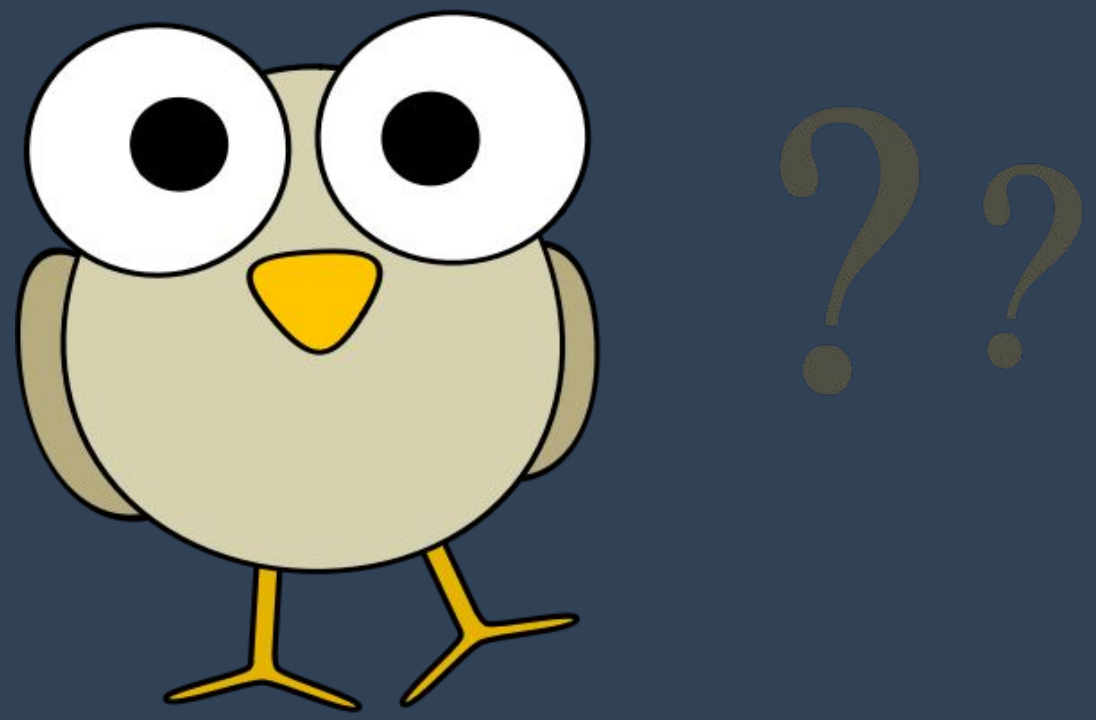
Classes & Objects

```python
# Defining classes
class Dog:
    def bark(self):
        print ("Woof!")

# Instantiating objects

my_dog = Dog ()
my_dog.bark()
# Output: Woof!
```

Inheritance

- **Definition:** A mechanism that allows one class to inherit properties and methods from another class

Can you explain the benefits of Inheritance?



Inheritance

- **Benefits:** Code reusability, reduced complexity

Single Inheritance

```python
class Animal:
    def speak(self):
        pass

class Dog (Animal):
    def speak(self):
        print ("Woof!")
```

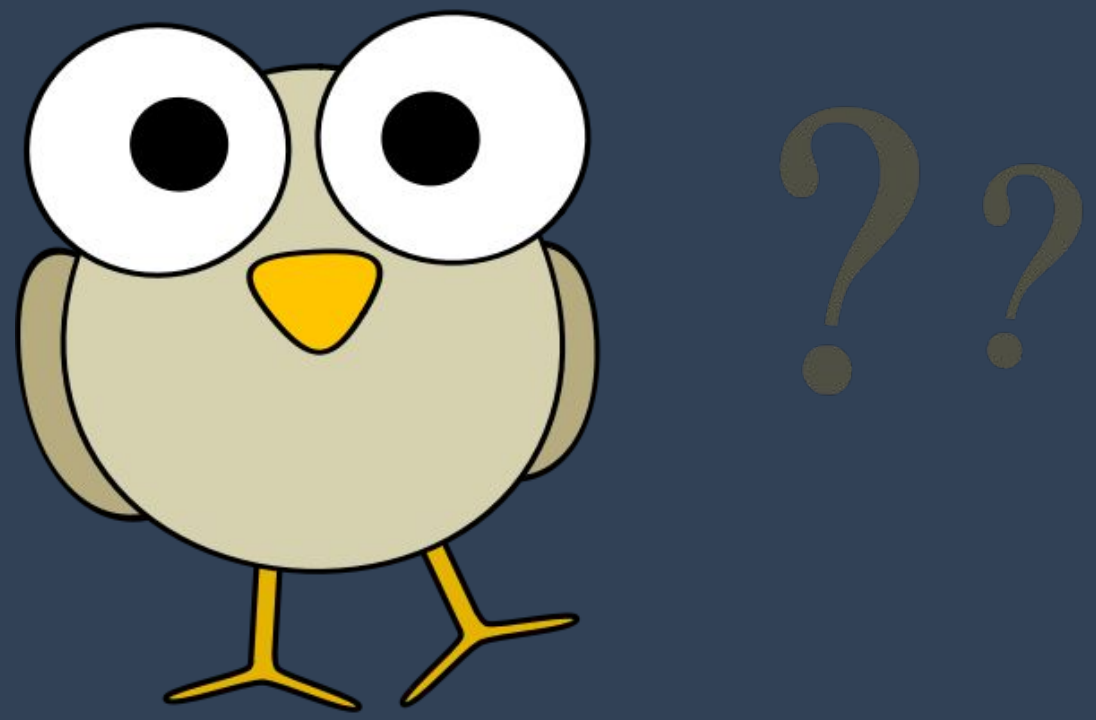
Multiple Inheritance

```python
class Bird:
    def fly(self):
        print ("I can fly!")

class Fish:
    def swim(self):
        print("I can swim!")


class FlyingFish(Bird, Fish):
    pass


ff = FlyingFish()
ff.fly()# Output: I can fly!
ff.swim()# Output: I can swim!
```

Encapsulation

- **Definition:** Bundling data and methods that operate on that data within a single unit (class)

Can you explain the benefits of Encapsulation?



Encapsulation

- **Benefits:** Improved data integrity, control over access to data

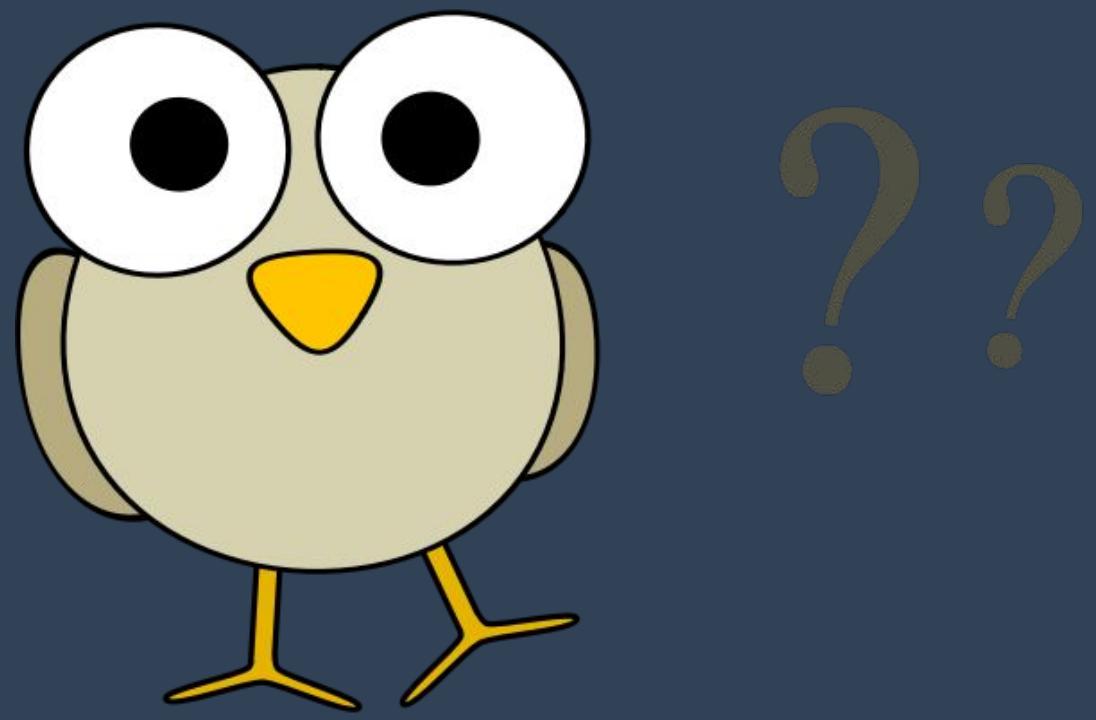
Encapsulation

```python
# Access specifiers
# Public, protected, and private attributes

class Car:
    wheels = 4 # Public attribute
    _color = "red" # Protected attribute
    __engine = "V8" # Private attribute
```

Polymorphism

- **Definition:** The ability of different classes to share the same interface and be treated as objects of a common superclass

Can you explain the benefits of Polymorphism?



Polymorphism

- **Benefits:** Flexibility in code, easier maintenance

Polymorphism

```python
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

calc = Calculator ()

print(calc.add(1, 2))# Output: 3

print(calc.add(1, 2, 3)) # Output: 6
```

Polymorphism

```python
class Animal:
    def speak(self):
    print ("Generic animal sound")
class Dog(Animal):
    def speak(self):
    print ("Woof!")

my_dog = Dog
```

Scope of Variables in Python

- **Scope:** The region of the code where a variable can be accessed and modified
- **Importance:** Proper scoping prevents unintended access or modification of variables

Local Variables

- **Local Variables:** Variables declared within a function or a code block
- Accessible only inside the function or block

Local Variables

```
def example_function():
    local var = 5
    print(local_var) # Output: 5

example_function ()
print (local_var)
# Error: NameError: name 'local var' is not defined
```

Nonlocal Variables

- **NonLocal Variables:** Variables defined in an outer function but not in the global scope
- Used in nested functions to modify the outer function's variable

Nonlocal Variables

```python
def outer function():
    nonlocal var = 10

    def inner_function():
        nonlocal nonlocal_var
        nonlocal_var += 5


    inner_function()
    print (nonlocal_var) # output: 15


outer_function()
```

Global Variables

- **Global Variables:** Variables declared outside any function or block
- Accessible from any part of the cod

Global Variables

```
global_var = 20
def example_function():
    print(global _var)# Output: 20


example_function ()
print (global_var) # Output: 20
```

Custom Sort Functions in Python

- **sorted():** Returns a new sorted list from the specified iterable
- **sort():** Modifies the list in-place and returns None

Custom Sort Functions in Python

```python
numbers = [ 4, 2, 9, 1, 5]
sorted_numbers = sorted(numbers)
print(sorted_numbers) # Output: [1, 2, 4, 5, 9]
print(numbers)# Output: [4, 2, 9, 1, 5]
numbers.sort()
print(numbers) # Output: [1, 2, 4, 5, 9]
```

Custom Sort Functions in Python

- **key:** Function to be called on each list element to determine the sorting order
- **reverse:** If set to True, the list is sorted in descending order

Custom Sort Functions in Python

main.py

main.py > ...

```python
word = ["apple", "banana", "cherry", "date", "fig"]
sorted_words = sorted(word, key=len, reverse=True)
print(sorted_words)
```

Output of Previous Code Snippet

```
Run                                                    138ms
['banana', 'cherry', 'apple', 'date', 'fig']
```

Exception Handling

- **Definition:** Events that occur during the execution of a program when an error is encountered

Exception Handling

- **Importance:** Proper handling prevents the program from crashing and allows for more graceful error management

Exception Handling

- Common Exceptions in Python
  - SyntaxError
  - TypeError
  - ValueError
  - FileNotFoundError

Python try...except Block

The `try...except` block is used to handle exceptions in Python. Here's the syntax of `try...except` block:

```
try:
    # code that may cause exception
except:
    # code to run when exception occurs
```

Here, we have placed the code that might generate an exception inside the `try` block. Every `try` block is followed by an `except` block.

When an exception occurs, it is caught by the `except` block. The `except` block cannot be used without the try block.

```
Syntax:

try:
# Some Code...
except:
# optional block
# Handling of exception (if required)
else:
# execute if no exception
finally:
# Some code  .. (always executed)
```

Raising Exceptions with raise

```python
def divide(a, b):
    if b == 0:
        raise ValueError ("Cannot divide by zero")
    return a / b


try:
    result = divide (10, 0)

except ValueError as e:
    print(e) # Output: Cannot divide by zero
```

Creating Custom Exceptions

```
class CustomError (Exception):
    def init _(self, message):
        self.message = message

try:
    raise CustomError ("This is a custom error!")
except CustomError as e:
    print (e.message) # Output: This is a custom error!
```

File Handling

- **Definition:** The process of reading from or writing to a file
- **Importance:** Persisting data, reading data from external sources, and generating reports

Different Modes to Open a File in Python

Mode	Description
`r`	Open a file for reading. (default)
`w`	Open a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
`x`	Open a file for exclusive creation. If the file already exists, the operation fails.
`a`	Open a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.

Opening Files

main.py input.txt +

main.py > ...

```python
file = open("input.txt", "r")
content = file.read()
print(content)
file.close()
```

Reading from Files

```python
with open("example.txt","r") as file:
    for line in file.readlines():
        print(line.strip())
```

- read(): Reads the entire content of the file
- readline(): Reads a single line from the file
- readlines(): Reads all lines and returns a list of lines

Writing to Files

- **write():** Writes a string to the file
- **writelines():** Writes a list of strings to the file

```
lines = ["First line\n", "Second line\n", "Third line\n"]
with open("output.txt","w") as file:
    file.writelines(lines)
```

JSON Module in Python

- **Definition:** JavaScript Object Notation (JSON) is a lightweight data-interchange format
- **Benefits:** Human-readable, language-independent, easy to parse and generate

JSON Module Functions

- **json.dump():** Serialize Python object to a JSON-formatted file
- **json.dumps():** Serialize Python object to a JSON-formatted string
- **json.load():** Deserialize JSON data from a file into a Python object
- **json.loads():** Deserialize JSON data from a string into a Python object

Working with JSON Data

```
import json
data = {
"name":
"John",
"age": 30,
"city": "New York"
}
json_string = json.dumps(data)
print(json_string)
```

Working with JSON Data

```
import json

json_string = '{"name": "John", "age": 30, "city": "New York")'
data = json.loads(json string)

print (data)
```

Working with JSON Data

```
import json
with open("data.json", "r") as file:
data = json. load(file)
print(data)
```

Working with JSON Data

```python
import json
data = {
"name": "John",
"age": 30,
"city": "New York"
}

with open("output.json", "w") as file:
    json.dump(data, file)
```

706. Design HashMap

Easy 4025 362 Add to List Share

Design a HashMap without using any built-in hash table libraries.

Implement the `MyHashMap` class:

- `MyHashMap()` initializes the object with an empty map.
- `void put(int key, int value)` inserts a `(key, value)` pair into the HashMap. If the `key` already exists in the map, update the corresponding `value`.
- `int get(int key)` returns the `value` to which the specified `key` is mapped, or `-1` if this map contains no mapping for the `key`.
- `void remove(key)` removes the `key` and its corresponding `value` if the map contains the mapping for the `key`.

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]
```

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]

Explanation
MyHashMap myHashMap = new MyHashMap();
myHashMap.put(1, 1); // The map is now [[1,1]]
myHashMap.put(2, 2); // The map is now [[1,1], [2,2]]
myHashMap.get(1);    // return 1, The map is now [[1,1], [2,2]]
myHashMap.get(3);    // return -1 (i.e., not found), The map is now [[1,1], [2,2]]
myHashMap.put(2, 1); // The map is now [[1,1], [2,1]] (i.e., update the existing
value)
myHashMap.get(2);    // return 1, The map is now [[1,1], [2,1]]
myHashMap.remove(2); // remove the mapping for 2, The map is now [[1,1]]
myHashMap.get(2);    // return -1 (i.e., not found), The map is now [[1,1]]
```

```python
class MyHashMap:

    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.myHashMap = dict()

    def put(self, key: int, value: int) -> None:
        """
        value will always be non-negative.
        """
        self.myHashMap[key] = value


    def get(self, key: int) -> int:
        """
        Returns the value to which the specified key is mapped, or -1 if this map contains no mapping
for the key
        """
        if key in self.myHashMap.keys():
            return self.myHashMap[key]
        return -1

    def remove(self, key: int) -> None:
        """
        Removes the mapping of the specified value key if this map contains a mapping for the key
        """
        self.myHashMap[key]=-1
```

JSON supports primitive types (strings, numbers, boolean), as well as nested arrays and objects. Simple Python objects are translated to JSON according to the following conversion:

Python	JSON
dict	object
list, tuple	array
str	string
int, long, float	number
True	true
False	false
None	null